

Q. 1 — Full Table Scans QUESTION

BritBooks has an orders table with one million rows. A query searches for one customer but there is no index on customer_id:

- **What the database may need to read:**
It needs to read all the rows for looking for the customer
- **Why the query becomes slower as the table grows:**
Because the number of row increase and has to read more row, and that is more time
- **One way to improve the query:**
Index the table by customer_id, so it knows where to search the customer id

Q. 2 — Indexing Basics QUESTION BritBooks frequently runs this query:

SELECT order_id, amount_gbp FROM orders WHERE customer_id = 101;

- **Which column should be indexed?**
By customer_id because it is a regular field used to filter, so if it is indexed the query can directly read the rows of particular customers.
- **How can the index improve the query?**
Indexing the query can directly read the rows of particular customers instead of reading all rows searching the customer.
- **What is one disadvantage of adding an index?**
The lecture is faster but the writing is slower, the new registers have to be inserted in the table and after registered in the index. And also the storage is higher.

Q. 3 — Practical: Observe a Scan and an Index

TASK

- **Create query_performance.db .**
- **Create the orders table.**
- **Insert 5,000 sample orders.**
- **Use EXPLAIN QUERY PLAN before creating an index.**
- **Create an index on Run customer_id.**
- **EXPLAIN QUERY PLAN again.**
- **Print both query plans.**

Exercise on week2_session5_lab.ipynb, here the code is only pasted

```
import sqlite3
import random
connection = sqlite3.connect("query_performance.db")
cursor = connection.cursor()
```

```

# Create and populate the table
cursor.execute("DROP TABLE IF EXISTS orders")
cursor.execute("""
    CREATE TABLE orders(
        order_id INTEGER PRIMARY KEY,
        customer_id INTEGER NOT NULL,
        amount_gbp REAL NOT NULL,
        status TEXT NOT NULL
    )
""")
import random

# Generar los datos
orders = []

for order_id in range(1, 5001):
    customer_id = random.randint(1, 1000)
    amount_gbp = round(random.uniform(5, 1000), 2)
    status = random.choice([
        "Pending",
        "Completed",
        "Cancelled"
    ])

    orders.append((order_id, customer_id, amount_gbp, status))

cursor.executemany("""
INSERT INTO orders (order_id, customer_id, amount_gbp, status)
VALUES (?, ?, ?, ?)
""", orders)

connection.commit()

print(f"{len(orders)} registros insertados.")

query = """
SELECT order_id, amount_gbp
FROM orders
WHERE customer_id = 3

```

```

"""
    5000 registros insertados.

# Check the plan before the index
cursor.execute(f"EXPLAIN QUERY PLAN {query}")
print("Before index:")
for row in cursor.fetchall():
    print(row)

    Before index:
    (2, 0, 216, 'SCAN orders')

# Create the index
cursor.execute("""
CREATE INDEX idx_orders_customer_id
ON orders(customer_id)
""")
connection.commit()

# Check the plan after the index
cursor.execute(f"EXPLAIN QUERY PLAN {query}")
print("After index:")
for row in cursor.fetchall():
    print(row)

    After index:
    (3, 0, 61, 'SEARCH orders USING INDEX idx_orders_customer_id (customer_id=?)')

```

Q. 4 — Practical: Filter Early

Create an SQLite script that compares these approaches:

Approach A:

- Read every order.
- Filter Dispatched rows in Python.
- Calculate the total in Python.

Approach B:

- Filter rows in SQL.
- Calculate the total using SUM .
- Return only the final value.

- Print both totals and confirm that they match.

Exercise on week2_session5_lab.ipynb, here the code is only pasted

```
# Approach A Pandas:
all_products_df = pd.read_sql_query("""SELECT * FROM ORDERS""", connection,
index_col="order_id")

dispatched_orders_df = all_orders_df[all_orders_df["status"]=="Dispatched"]
python_total_amount = dispatched_orders_df["amount_gbp"].sum()
python_total_amount
```

```
np.float64(650944.29)
```

```
# Approach B
cursor.execute("""SELECT SUM(amount_gbp) FROM ORDERS WHERE status =
'Dispatched'""")
sql_total = cursor.fetchall()
sql_total
```

```
[(650944.29,)]
```

Q. 5 — MCQ: Full Table Scan

What happens during a full table scan?

- A. The database reads only one indexed row.
- B. The database checks every row in the table.
- C. The database deletes unused rows.
- D. The database compresses the table.

Q. 6 — MCQ: Index Choice

Which column is the best index candidate for this common query?

SELECT * FROM orders WHERE customer_id = 101;

- A. customer_id
- B. amount_gbp
- C. A column that is never queried
- D. Every column automatically

Q. 7 — MCQ: Index Trade-off

What is a common disadvantage of an index?

- A. It prevents all queries.
- B. It uses storage and adds write overhead.
- C. It removes primary keys.
- D. It converts rows into files

Q. 8 — MCQ: Filtering Early

Which query transfers the least unnecessary data?

- A. SELECT * FROM orders;
- B. SELECT * FROM orders; followed by Python filtering
- C. SELECT order_id FROM orders WHERE status = 'Dispatched';
- D. SELECT * FROM customers, orders;

Q. 9 — MCQ: Low-Cardinality Columns

An order_status column contains only Confirmed , Dispatched , and Cancelled. Why might an index on this column provide limited improvement for some queries?

- A. The column has very few distinct values.
- B. Text columns can never be indexed.
- C. Indexes work only on dates.
- D. Status values are stored in pounds.

Q. 10 — Partitioning Fundamentals

BritBooks stores three years of order data. Most reports filter by order date.

Explain:

- **A suitable partition key:** date
- **A possible folder layout:**
 - Orders
 - 2026-07-17
 - 2026-07-16
 - 2026-07-15
- **How partition pruning reduces the amount of data read:** the query goes to read the data into the partitions that it needs, and the data that is in partitions doesn't need it.

Q. 11 — Design a Partition Scheme

BritBooks has 100 million orders per year. Reports that normally filter by month. Very few reports for a single hour.

Choose one layout and justify your decision:

- A. /orders/year=2026/month=07/
- B. /orders/year=2026/month=07/day=17/hour=10/

Option A, because the usual grain is months, so having partitions with months will be faster than having partitions per month, day and hour. In that case there would be a lot of different files.

Q. 12 — The Small-Files Problem

A daily partition contains 10,000 files, each approximately 50 KB.

- **Why opening many small files can be inefficient**
Because all has to be opened, read the metadata and closed.
- **What compaction means**
Group all those small files into a larger one.
- **When a compaction job could run**
After the files load and before the transformation, once the day finish.

Q. 13 — Practical: Combine Small CSV Files

BritBooks receives three small order files for one day.

- orders_1.csv
- orders_2.csv
- orders_3.csv

Each file contains: order_id,customer_name,amount_gbp

Tasks:

- Create a folder named small_order_files .
- Create the three CSV files with one order in each.
- Combine them into orders_2026-07-17.csv .
- Write the header only once.
- Print the number of combined rows.

Exercise on week2_session5_lab.ipynb, here the code is only pasted

```
from pathlib import Path
import pandas as pd

# Folder creation
new_folder_path = Path("Folder for session5/small_order_files")
new_folder_path.mkdir(exist_ok=True)

# CSVs creation
columns_name = ["order_id", "customer_name", "amount_gbp"]
orders1 = [[1, "Vicky", 91.99]]
orders2 = [[2, "Alex", 45.45]]
```

```

orders3 = [[3, "Martin", 25]]

orders1_df = pd.DataFrame(orders1, columns=columns_name)
orders2_df = pd.DataFrame(orders2, columns=columns_name)
orders3_df = pd.DataFrame(orders3, columns=columns_name)

orders1_df.to_csv('Folder for session5/small_order_files/orders1.csv',
index=False, encoding='utf-8')
orders2_df.to_csv('Folder for session5/small_order_files/orders2.csv',
index=False, encoding='utf-8')
orders3_df.to_csv('Folder for session5/small_order_files/orders3.csv',
index=False, encoding='utf-8')

# CSVs read
o1_read=pd.read_csv('Folder for session5/small_order_files/orders1.csv',
header=0)
o2_read=pd.read_csv('Folder for session5/small_order_files/orders2.csv',
header=0)
o3_read=pd.read_csv('Folder for session5/small_order_files/orders3.csv',
header=0)

# Combine the CSVs
combined_df = pd.concat([o1_read, o2_read, o3_read])
combined_df

```

	order_id	customer_name	amount_gbp
0	3	Martin	25.00
0	2	Alex	45.45
0	3	Martin	25.00

```

#create combined CSV
combined_df.to_csv('Folder for session5/small_order_files/orders_2026-07-17.csv',
index=False, encoding='utf-8')

```

Q. 14 — MCQ: Partition Key

Which is the best partition key when most queries filter orders by month?

- A. Customer name
- B. Order month
- C. Product description
- D. Order amount rounded to the nearest pound

Q. 15 — MCQ: Partition Pruning

What does partition pruning do?

- A. Deletes old partitions automatically
- B. Reads only partitions that match the filter
- C. Creates an index on every column
- D. Converts CSV into JSON

Q. 16 — MCQ: Over-Partitioning

What is a possible result of partitioning data too finely?

- A. Too many small folders and files
- B. All rows become duplicates
- C. SQL stops supporting filters
- D. Compression becomes impossible

Q. 17 — MCQ: Small Files

Why are thousands of tiny files often inefficient?

- A. They cannot contain numbers.
- B. Each file adds metadata and opening overhead.
- C. They always delete their schemas.
- D. Small files cannot be stored in folders.

Q. 18 — MCQ: Compaction

What is compaction?

- A. Splitting one file into thousands of files
- B. Combining small files into fewer larger files
- C. Removing all partition columns
- D. Replacing SQL with Python

Q. 19 — Compare Common File Formats

Complete the table using short phrases.

Format	Text or binary?	Row or column oriented?	Common use
CSV	Text	Row	Small files in raw

			data
Parquet	Binary	Column	Data lakes and analytics
Avro	Binary	Row	Streaming and raw data
ORC	Binary	Column	Hive and Hadoop analytics

Q. 20 — Row and Columnar Storage

BritBooks has an analytics file with 80 columns. A report needs only: `order_date`, `product_id` and `amount_gbp`. Explain why a columnar format may read less data than CSV.

Columnar format searches directly those columns and reads them. In a row format it has to read all the rows to search those columns.

Q. 21 — Choose a File Format

Choose the most suitable option and give one reason:

Requirement	Choose from	Reason
A small file that a non-technical user can open easily	CSV	It is human readable with plain text
Large analytical data where queries select a few columns	Parquet	Parquet allows to read by columns
Streaming events with an explicit schema	Avro	It has a flexible schema and can store serial data
Large Hive analytics tables	ORC	It command use in Hive, it has a columnar format

Q. 22 — Practical: Create an Avro-Style Schema File

BritBooks sends order events between systems.

REQUIRED FIELDS:

Field	Type
order_id	string
customer_id	string
amount_gbp	double
order_status	string
coupon_code	null or string, default null

TASK:

- Create the schema as a Python dictionary.
- Write it to order_event_schema.json .
- Use indentation to keep the JSON readable.
- Read the file again.
- Print the field names.
-

Exercise on week2_session5_lab.ipynb, here the code is only pasted

```
import json
# Schema as python dictionary
schema = {
    "name": "order_events",
    "type": "record",
    "fields": [
        {
            "name": "order_id",
            "type": "string"
        },
        {
            "name": "customer_id",
            "type": "string"
        },
        {
            "name": "amount_gbp",
            "type": "double"
        },
        {
            "name": "order_status",
            "type": "string"
        },
        {
            "name": "coupon_code",
            "type": "string",
            "default": null
        }
    ]
}
```

```

        "name": "order_status",
        "type": "string"
    },
    {
        "name": "customer_id",
        "type": "string"
    },
    {
        "name": "coupon_code",
        "type": ["string", "null"],
        "default": None
    }
]
}
# Write schema into a json
with open("Folder for session5/small_order_files/order_event_schema.json", "w",
encoding="utf-8") as file:
    json.dump(schema, file, indent=4)
# Read the json
with open("Folder for session5/small_order_files/order_event_schema.json", "r",
encoding="utf-8") as file:
    json_schema_file = json.load(file)

json_schema_file

```

```

{'name': 'order_events',
 'type': 'record',
 'fields': [{'name': 'order_id', 'type': 'string'},
 {'name': 'customer_id', 'type': 'string'},
 {'name': 'amount_gbp', 'type': 'double'},
 {'name': 'order_status', 'type': 'string'},
 {'name': 'customer_id', 'type': 'string'},
 {'name': 'coupon_code', 'type': ['string', 'null'], 'default': None}]}

```

```

# Read the files
json_schema_fields = [field["name"] for field in json_schema_file["fields"]]
json_schema_fields

```

```
['order_id',  
'customer_id',  
'amount_gbp',  
'order_status',  
'customer_id',  
'coupon_code']
```

Q. 23 — MCQ: CSV

Which statement about CSV is correct?

- A. It always stores a strict schema.
- B. It is a human-readable text format.
- C. It is a columnar binary format.
- D. It automatically supports nested objects.

Q. 24 — MCQ: Parquet

Why is Parquet commonly used for analytics?

- A. It stores every value as plain text.
- B. It is columnar and can read selected columns.
- C. It supports only one data type.
- D. It cannot be compressed

Q. 25 — MCQ: Avro

Which use case best matches Avro?

- A. A simple spreadsheet opened manually
- B. Structured event exchange with schema evolution
- C. Image storage
- D. Graph traversal

Q. 26 — MCQ: ORC

ORC is mainly associated with:

- A. Columnar analytics in the Hadoop and Hive ecosystem
- B. Storing passwords
- C. Editing source code
- D. Relational transactions

Q. 27 — MCQ: Compression

What is the main benefit of compression?

- A. It always removes CPU usage.
- B. It reduces storage size and data read from disk.
- C. It removes the need for a schema.

D. It converts columns into rows.

Q. 28 — Identify the Requirements

BritBooks needs a daily sales report.

The report:

- **Filters by order month**
- **Groups by product**
- **Reads only order date, product ID, quantity, and amount**
- **Uses data stored for five years**

Identify:

- **The best partition field:** year,month
- **A suitable analytical file format:** Parquet
- **The columns the query should select :** SELECT order_date, product ID, quantity and amount
- **One reason to avoid SELECT * :** Reads all the columns, the ones needed and no

Q. 29 — Create a Storage Decision Table

BritBooks has these datasets:

- **Raw order exports**
- **Cleaned analytical orders**
- **Streaming click events**
- **Monthly sales summaries**

TASK Create a table with:

Dataset	Type format	Partition key	Reason
Raw order exports	AVRO	Date	Avro is lighter than file formats as csv
Cleaned analytical orders	Parquet	Date	Structured data in a columnar format, has high performance
Streaming click events	AVRO	Date and event	Avro is indicated for streaming data and has a flexible schema
Monthly sales summaries	Parquet	Date, month	Structured data in a columnar format, has high performance

Q. 30 — Practical: Create a Partitioned Folder Layout
BritBooks wants to organise orders by year and month.

1001, 2026-06-28, £34.99

1002, 2026-07-02, £18.50

1003, 2026-07-17, £42.75

- Create an orders_lake folder.
- Create year=YYYY/month=MM folders from each date.
- Write each order to an orders.csv file in the correct partition.
- Include the CSV header.
- Print the created file paths.

Exercise on week2_session5_lab.ipynb, here the code is only pasted

```
from pathlib import Path
from datetime import datetime
import csv

# Orders lake folder creation
order_lake_path = Path("Folder for session5/order_lake")
order_lake_path.mkdir(exist_ok=True)

orders = [
    {"order_id": "1001", "order_date": "2026-06-28", "amount_gbp": 34.99},
    {"order_id": "1002", "order_date": "2026-07-02", "amount_gbp": 18.50},
    {"order_id": "1003", "order_date": "2026-07-17", "amount_gbp": 42.75}
]

# Loop for creating directories, csv file and the data
for order in orders:
    date = datetime.strptime(order["order_date"], "%Y-%m-%d").date()
    header = order.keys()

    folderpath_year_month = (
        order_lake_path /
        f"{date.strftime('%Y')}" /
        f"{date.strftime('%m')}"
    )
    folderpath_year_month.mkdir(exist_ok=True, parents=True)
```

```

csv_path = folderpath_year_month / "orders.csv"

if csv_path.exists():
    mode = "a"
else:
    mode = "w"

with open(csv_path, mode , encoding="utf-8") as file:
    writer = csv.writer(file, lineterminator='\n')
    if mode=="w":
        writer.writerow(order.keys())
        writer.writerow(order.values())

for file_path in sorted(order_lake_path.rglob("orders.csv")):
    print(file_path)

```

```

Folder for session5\order_lake\2026\06\orders.csv
Folder for session5\order_lake\2026\07\orders.csv

```

Q. 32 — MCQ: Analytics Format

Which format is the strongest default choice for large analytical order data?

- A. Parquet
- B. Plain text without separators
- C. Image files
- D. HTML

Q. 33 — MCQ: Partition Filter MCQ

Which filter is most likely to support pruning for this layout?

/orders/year=2026/month=07/

- A. WHERE customer_name = 'Amelia Clarke'
- B. WHERE year = 2026 AND month = 7
- C. WHERE amount_gbp > 20
- D. WHERE status = 'Confirmed'

Q. 34 — MCQ: Projection

What does projection mean in query optimisation?

- A. Selecting only required columns

- B. Deleting old rows
- C. Creating additional small files
- D. Sorting every table

Q. 35 — MCQ: Streaming Events

Which format is a suitable choice for structured streaming event records?

- A. Avro
- B. Bitmap image
- C. PDF D.
- Plain HTML

Q. 36 — MCQ: Combined Optimisation

Which combination is most suitable for monthly sales analytics?

- A. One large CSV, SELECT * , and no date filter
- B. Parquet, date partitions, early filtering, and required columns only
- C. Thousands of tiny text files and no compression
- D. Graph storage for every sales total